

Credit Card Approval Prediction System

FSD Documentation

1. Introduction

Project Title

Credit Card Approval Prediction System

Team Members

List each team member's name and their role below (e.g. Data Collection & Preprocessing, Model Building, Web Application Development, Deployment & Testing).

- P. Chandana Sri — Data Collection & Preprocessing
- K. Hannah — Model Building & Evaluation
- K. Nandu Priya — Web Application (Flask) Development
- P. Neelofer — Deployment & Testing

2. Project Overview

Purpose

The project automates credit card risk assessment. Instead of a slow, manual review of an applicant's income, employment and credit history, the system takes the applicant's details through a web form and instantly predicts whether they are a good (Approved) or bad (Rejected) credit risk using a trained machine-learning model.

Features

- Simple web form to capture personal, financial, family and credit-history details.
- Automated preprocessing and encoding of applicant inputs.
- Comparison of multiple ML models (Logistic Regression, Decision Tree, Random Forest, XGBoost) and automatic selection of the best one by F1-score.
- Instant Approved / Rejected prediction with a clear result page.
- Lightweight, stateless design that can be deployed and demoed live on the web.

3. Architecture

This project uses a Python-based web stack rather than the MERN stack — Flask on the backend, server-rendered HTML/CSS/JS on the frontend, and a trained model artifact in place of a traditional database.

Frontend

Built with HTML, CSS and JavaScript, rendered via Flask's Jinja2 templates (no separate React app). The form is grouped into sections — Personal, Financial, Family and Credit History — and the result page displays a clear Approved/Rejected message.

Backend

Built with Flask (Python). The backend exposes routes to serve the form, receive submissions, preprocess the data, run the trained model, and return the prediction. Gunicorn serves the app in production.

Database

There is no traditional database (e.g. MongoDB) in the current version. Instead, the system uses:

- Trained model artifact — model.pkl (best of Logistic Regression, Decision Tree, Random Forest, XGBoost).
- Encoders artifact — encoders.pkl (label encoders fit during training, reused at inference time).
- Historical training data — application_record.csv and credit_record.csv, used offline to train the model, not queried at request time.

4. Setup Instructions

Prerequisites

- Python 3.9+
- pip
- Flask, scikit-learn, XGBoost, pandas, numpy, gunicorn (see requirements.txt)

Installation

```
git clone <repository-url>
cd credit-card-approval-prediction
pip install -r requirements.txt
```

No external environment variables (e.g. database URIs or API keys) are required for the current version, since the model is loaded from local pickle files.

5. Folder Structure

Client (Templates & Static Assets)

There is no separate React client. The 'frontend' lives inside the Flask app:

- templates/ — HTML pages (landing page, form page, result page) rendered by Flask.
- static/ — CSS and JavaScript assets used by the templates.

Server

- app.py — Flask application: routes, preprocessing, and prediction logic.
- model/ — model.pkl and encoders.pkl, the trained artifacts loaded at startup.
- notebooks/ — EDA, preprocessing and model-training notebooks used to produce the artifacts.
- requirements.txt — Python dependencies.
- Procfile / render.yaml — deployment configuration for Render.

6. Running the Application

Since the frontend is served by Flask itself, there is a single command to run the whole application locally:

```
python app.py
```

The app will be available at <http://localhost:5000> (or the port specified in app.py). There is no separate 'npm start' for client and server, as this is not a MERN-style split-client application.

7. API Documentation

Endpoints exposed by the Flask backend:

Method	Endpoint	Description	Example Response
GET	/	Renders the landing page describing the tool.	HTML page
GET	/form	Renders the application form (personal, financial, family, credit-history fields).	HTML form page
POST	/predict	Accepts submitted form data, preprocesses & encodes it, runs the trained model, and returns the risk decision.	{ "prediction": "Approved" } or { "prediction": "Rejected" }

8. Authentication

The current version does not implement user authentication or sessions — the prediction form is public and stateless; each request is processed independently with no login required. Adding applicant accounts, admin login (for analysts), and token-based authentication (e.g. JWT) is listed under Future Enhancements.

9. User Interface

Add screenshots or GIFs here showing:

- The landing page introducing the tool.
- The multi-section application form (Personal, Financial, Family, Credit History).
- The Approved / Rejected result page.

10. Testing

Testing combines standard ML model evaluation with manual application testing:

- Model evaluation: train/test split, F1-score comparison across Logistic Regression, Decision Tree, Random Forest and XGBoost; confusion matrix review.
- Form validation testing: checking required fields, invalid ranges (e.g. negative income), and edge cases.
- Manual end-to-end testing: submitting sample applicant profiles and verifying the correct Approved/Rejected page is shown.

11. Screenshots or Demo

Add screenshots of the running application here, or a link to the live demo (e.g. the Render-hosted URL).

- Live demo: [Add Render deployment URL here]

12. Known Issues

- No user authentication — anyone with the link can submit predictions.
- No explanation of which specific factors drove an Approved/Rejected decision (limited explainability).
- Class imbalance in the training data (few 'risky' examples) can bias the model toward predicting Approved.
- No persistent storage of past applications or predictions for later review.

13. Future Enhancements

- Add applicant accounts and admin/analyst login with authentication.
- Add explainability (e.g. SHAP values) to show applicants which factors affected their result.
- Store applications and predictions in a database for audit and analyst review.
- Add a retraining pipeline to periodically update the model on new data.
- Add a human-review/appeal workflow for borderline or rejected applicants.